

OS UNIT-4 ORANGE PROBLEM

Screenshots:

1A)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$ make test_objects
./test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$
```

1B)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$
```

2A)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$ make test_tree
make: 'test_tree' is up to date.
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$
```

2B)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                ES-VCS!.
```

3A)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index      # Human-readable text format
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
Staged changes:
  staged:    file1.txt
  staged:    file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked: .gitignore
  untracked: Makefile
  untracked: README.md
  untracked: commit.c
  untracked: commit.h
  untracked: index.c
  untracked: index.h
  untracked: object.c
  untracked: pes.c
  untracked: pes.h
  untracked: test_objects
  untracked: test_objects.c
  untracked: test_sequence.sh
  untracked: test_tree
  untracked: test_tree.c
  untracked: tree.c
  untracked: tree.h

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776780390 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776780390 6 file2.txt
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$
```

3B)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776780390 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776780390 6 file2.txt
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$
```

4A)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$ ./pes log
commit 63167a1cc7e392c0013ebffa944e88508f9c335fb2f6da4088749761717f0b2b
Author: Karnati Uday Keerthan Reddy <PES1UG24CS217>
Date: 1776780504

    Add farewell

commit 021123d7bb0f3441550cdbf8c7ac1c7c58b3096d4042f056ac13727f217b5958
Author: Karnati Uday Keerthan Reddy <PES1UG24CS217>
Date: 1776780504

    Add world

commit 3896df5265cddfcc8a6187c6c5adc4cf3c7f244dd3c516b8881083bf34b819ed
Author: Karnati Uday Keerthan Reddy <PES1UG24CS217>
Date: 1776780504

    Initial commit farewell
```

4B)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/02/1123d7bb0f3441550cdbf8c7ac1c7c58b3096d4042f056ac13727f217b5958
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdec7e3f3a183d337639025de576db9ebb4
.pes/objects/38/96df5265cddfcc8a6187c6c5adc4cf3c7f244dd3c516b8881083bf34b819ed
.pes/objects/63/167a1cc7e392c0013ebffa944e88508f9c335fb2f6da4088749761717f0b2b
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/8a/a0f9001b8afb415ac235a1e81c539b1809f3e0b67264ae7ec7431902a7f39d
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/9d/07fa9ae5e5cdc4298248d7bf30a9145c9adec0661c98a4f06aeaddbf7b64ca
.pes/objects/bc/86fd0e9fe731e48d5d6951259706bd06b494e121d01576a3f6fc67f31350b5
.pes/objects/c9/6f2fc25394979be592d5cf80c5d78ec792ff13dde17f838449fbcdffb7df0d
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$
```

4C)

```
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$ cat .pes/refs/heads/main
63167a1cc7e392c0013ebffa944e88508f9c335fb2f6da4088749761717f0b2b
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine:~/Desktop/PES1UG24CS217-pes-vcs$
```

make test-integration

```

udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$ make test-integration
=== Running integration tests ===
bash test sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
    staged:   file.txt
    staged:   hello.txt

Unstaged changes:
(nthing to show)

Untracked files:
(nthing to show)

--- First Commit ---
Committed: fe06d3545a97... Initial commit

Log after first commit:
commit fe06d3545a97bad4c9105632b4ef138809dd789f2e928f80cd09ae66dfc9d65f
Author: Karnati Uday Keerthan Reddy <PES1UG24CS217>
Date:   1776780826

    Initial commit

--- Second Commit ---
Committed: 81643b26e517... Update file.txt

--- Third Commit ---
Committed: ab40f812fcdce0400ff523e0f14b20ee9e4bd75e4d88fc1077d1acbf4b987cf2

--- Full History ---
commit ab40f812fcdce0400ff523e0f14b20ee9e4bd75e4d88fc1077d1acbf4b987cf2
Author: Karnati Uday Keerthan Reddy <PES1UG24CS217>
Date:   1776780826

    Add Farewell

commit 81643b26e517c31d71952145adcea364526e669b7a0977606e9d2620a6341d37
Author: Karnati Uday Keerthan Reddy <PES1UG24CS217>
Date:   1776780826

    Update file.txtfarewell

commit fe06d3545a97bad4c9105632b4ef138809dd789f2e928f80cd09ae66dfc9d65f
Author: Karnati Uday Keerthan Reddy <PES1UG24CS217>
Date:   1776780826

    Initial committial1

--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
ab40f812fcdce0400ff523e0f14b20ee9e4bd75e4d88fc1077d1acbf4b987cf2

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd0b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a1227423d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/81/643b26e517c31d71952145adcea364526e669b7a0977606e9d2620a6341d37
.pes/objects/ab/40f812fcdce0400ff523e0f14b20ee9e4bd75e4d88fc1077d1acbf4b987cf2
.pes/objects/ab/5824a9ec1ef585b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e00f77fa1974d901ce282f30e08de8cccd92
.pes/objects/d0/7723c25dd2d137b7574be085542b5620f40bafeaa3829f61f75b0d1d05350f0
.pes/objects/db/07a1451ca9544dbf66d760b505377a765afe7adcc607b75cc9d2b3b3d6eff
.pes/objects/fe/06d3545a97bad4c9105632b4ef138809dd789f2e928f80cd09ae66dfc9d65f

=== All integration tests completed ===
udayk24ubuntu@udayk24ubuntu-QEMU-Virtual-Machine: ~/Desktop/PES1UG24CS217-pes-vcs$

```

Phase 5 & 6: Analysis-Only Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout — what files need to change in

.pes/, and what must happen to the working directory? What makes this operation complex?

A branch is stored as a file in `.pes/refs/heads/<branch>` containing a commit hash. To implement `pes checkout <branch>`, first read the commit hash from the branch file and update `.pes/HEAD` to point to that branch (`ref: refs/heads/<branch>`). Then load the commit object, extract its tree, and reconstruct the working directory to match that tree by creating, modifying, or deleting files accordingly using blob data. The `.pes/index` must also be updated to reflect this new snapshot. This operation is complex because it must carefully overwrite the working directory without losing uncommitted changes, handle nested directory structures (trees), and maintain consistency between HEAD, index, and working directory.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

To detect a dirty working directory, compare the current working directory with the index. For each tracked file, check metadata such as modification time and file size; if they differ from the index, the file is modified. Then compare the index version of the file with the version in the target branch's tree using blob hashes. If a file is modified in the working directory and also differs between the current branch and the target branch, it creates a conflict. In this case, checkout must be refused to prevent overwriting uncommitted changes.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In a detached HEAD state, new commits are created normally but are not referenced by any branch. This means they are not part of the main commit history and can become unreachable if the user switches branches. These commits may eventually be removed by garbage collection. To recover them, the user can create a new branch pointing to the current commit before switching away, or manually note the commit hash and create a branch later using that hash.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

A mark-and-sweep algorithm can be used. In the mark phase, start from all branch heads in `.git/refs/heads/` and traverse all reachable commits, trees, and blobs using DFS or BFS, marking each visited object. A hash set should be used to store reachable hashes efficiently for constant-time lookup. In the sweep phase, iterate through all objects in `.git/objects/` and delete those not present in the reachable set. For a repository with 100,000 commits, including trees and blobs, approximately 200,000 to 300,000 objects may need to be visited.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete

an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Running garbage collection concurrently with a commit is dangerous because of race conditions. For example, a commit operation may write new objects (blobs, trees, commit) but has not yet updated the branch reference. At the same time, garbage collection scans the object store and does not see these new objects as reachable, so it deletes them. This results in a broken commit referencing missing objects. Git avoids this by using mechanisms like locking, temporary files, and delaying deletion (e.g., grace periods) to ensure that objects being written are not removed during garbage collection.